

Compiler-Exempt Sensitivity Handling (CESH)

Role-Aware Layer-Specific Quantization Policy with DIGS Gradient Sculpting and Low-Rank Embedding Decomposition in NeuralScript

Authors: Brandon Wiemer Date: April 2026 Status: Research Proposal

Abstract

We propose CESH (Compiler-Exempt Sensitivity Handling), a compile-time role-aware quantization policy for NSL that codifies a well-established but under-formalized industry practice: embedding and output-head layers require fundamentally different quantization treatment than transformer body layers. CESH introduces: (1) layer-role annotation as a first-class compiler property, inferred from position in the model graph or explicitly user-declared; (2) role-specific default policies that exempt embedding and head layers from the most aggressive CFTP/CFIP assignments while allowing body layers full aggressiveness; (3) a port of DIGS (Decomposition-Informed Gradient Sculpting, developed for CodeForge's TCFP-12) to NSL, providing tokenizer-aware gradient sculpting at the output head; (4) optional low-rank decomposition of the embedding matrix for models where the $V \times d$ matrix dominates memory; (5) outlier-safe training hooks integrated with the Muon optimizer and Single-Scale RMSNorm research line. CESH addresses the single-largest class of quantization accuracy failures in LLM deployment: the one where the user applied uniform W4A4 to a model and accuracy collapsed because the embedding layer or LM head lost resolution. Expected impact: order-of-magnitude fewer quantization accuracy failures, 15-25% embedding memory reduction via optional low-rank factorization, and near-lossless 4-bit inference on models where naive uniform quantization fails catastrophically.

1. The Embedding/Head Problem

1.1 Why these layers are special

Embedding and output-head layers have statistically different properties from transformer body layers:

Size. For a model with vocabulary V and dimension d :

- Embedding matrix: $V \times d$ parameters.
- Output head: $d \times V$ parameters (often tied to the embedding; sometimes not).

Concrete sizes:

Model	V	d	Embed + head	% of total
Llama-3-8B	128,256	4,096	1.05 B	13%
Llama-3-70B	128,256	8,192	2.10 B	3%
Phi-3-Mini	32,064	3,072	0.20 B	5%
Gemma-7B	256,000	3,072	1.57 B	20%

For smaller models with large vocabularies (Gemma-7B: 20% of weights are embedding+head), these layers dominate memory. Quantizing them aggressively matters for compression; quantizing them badly matters for accuracy.

Usage asymmetry.

- Embedding matrix: read-only per token (row lookup). Each row read once per appearance. Rows follow Zipfian frequency (top 100 tokens dominate usage).
- Output head: read in full for every forward pass (the entire $d \times V$ matmul produces logits for every token).

Outlier distribution. LLM.int8() established that outlier features — feature dimensions with values 10-100× larger than median — emerge at scale (>6.7B parameters) and are *crucial* for model performance. Setting outlier features to zero drops performance by 20%+. These outliers appear in the residual stream, meaning every layer's output has them — but the embedding layer *originates* them. The output head, projecting the final residual stream to vocabulary, also interacts with outliers unusually.

Sensitivity. Embedding and LM head layers are systematically the most sensitive to quantization in standard architectures. The AutoQuantize (NVIDIA Model Optimizer) gradient-based sensitivity score consistently ranks these layers as "keep in higher precision."

1.2 Existing industry practice

Every major LLM quantization pipeline treats these layers specially:

- **AWQ**: keeps first and last layers at FP16 by default.
- **GPTQ**: applies per-layer calibration; embedding and head layers often end up higher bit budget.
- **LLM.int8()**: mixed-precision decomposition keeps outlier feature dimensions at FP16, which disproportionately affects embedding and head.
- **NVIDIA Model Optimizer**: AutoQuantize's gradient-based sensitivity score flags these layers for higher bit budget.
- **SmoothQuant, QuaRot, SpinQuant**: use rotation tricks (Hadamard, learned) to redistribute outliers before quantization. Applied uniformly but disproportionately benefit the outlier-heavy layers.

The practice is well-established; the formalization in a compiler is missing.

1.3 Why NSL needs this

NSL currently has CFTP, CFIP, and CTQS — all treat all layers uniformly modulo WGGO's per-layer decisions. WGGO has no sensitivity awareness built in; it applies uniform policies unless explicitly overridden.

Additionally, CodeForge's **DIGS** (Decomposition-Informed Gradient Sculpting) was developed for TCFP-12 but not ported to NSL. DIGS uses the W_{hi}/W_{lo} residual magnitude from dual-GEMM decomposition to sculpt gradients — particularly effective at the output head where tokenizer merge-table structure creates predictable decomposition-hostile directions.

CESH formalizes the industry practice as an NSL compiler feature and ports DIGS as a specific instantiation for the output head.

2. Six Compiler-Native Sensitivity Innovations

2.1 Innovation 1: Layer-role as a first-class property

The NSL type system gains a `@role` attribute on layers:

```
ns!

@role("embedding")
embedding_layer: Embedding<[vocab_size, d_model]>

@role("body")
transformer_blocks: [TransformerBlock; num_layers]

@role("lm_head")
output_projection: Linear<[d_model, vocab_size]>
```

Role values: `"embedding"`, `"lm_head"`, `"body"` (default), `"auxiliary"` (for layers like router networks that should follow body policy but get separate treatment).

When the role is not explicitly annotated, CESH infers it:

- First layer receiving token IDs as input → `"embedding"`.
- Last layer producing logits (output shape `[*, vocab_size]`) → `"lm_head"`.
- Tied embedding/head (a common pattern) → both layers get the shared role inferred from position.
- Everything else → `"body"`.

Inference is conservative: if role is ambiguous, the user gets a warning and body policy is applied.

2.2 Innovation 2: Role-specific default policies

CESH defines per-role defaults in WGGO's decision-making:

Embedding layer defaults:

- Keep weights at FP16 unless `@quant_override` explicitly requests lower.
- Embedding lookup (row-wise) does not involve matmul, so quantization-accelerated kernels don't benefit; FP16 is essentially free.
- If memory pressure dictates compression, prefer low-rank factorization (see §2.5) over bit-width reduction.

LM head defaults:

- Weights at FP16 or FP8 (not 4-bit by default).
- Activations at FP16 (never quantized at this boundary).
- DIGS gradient sculpting active during training (see §2.4).
- If quantized, apply SmoothQuant or QuaRot-style rotation before quantization.

Body defaults:

- Full CFTP + CFIP aggressiveness permitted.
- CTQS phase transitions can push body layers to very low bit budgets (2-bit, even ternary).
- No rotation tricks by default (body layers handle their own outliers through GEBA).

User override is always possible via explicit `@quant_override` annotations. The defaults just set what happens in the absence of explicit direction.

2.3 Innovation 3: Outlier-aware weight rotation for LM head

When the LM head is to be quantized below FP16, CESH applies a Hadamard rotation (QuaRot-style) to the weights and the residual stream input:

ns!

```
// CESH-transformed LM head (at compile time)
@role("lm_head")
output_projection:
  # Rotation pre-applied to weights (offline)
  W_rot = H^T @ W

  # At inference time:
  def forward(x):
    x_rot = H^T @ x      # fused into the RMSNorm before output
    logits = x_rot @ W_rot
    return logits
```

The Hadamard matrix H is a deterministic structured orthogonal matrix; the rotation is mathematically equivalent to the original computation but redistributes the outliers. After rotation, naive 4-bit quantization of the rotated weights loses far less accuracy than quantization of the original weights.

This is implemented as a compile-time weight transformation. No runtime cost (the rotation of activations is fused into the preceding RMSNorm); inference latency is unchanged.

2.4 Innovation 4: DIGS port for the output head

DIGS (Decomposition-Informed Gradient Sculpting) was developed for CodeForge's TCFP-12 system. Core insight: TCFP's dual-GEMM decomposition ($W = W_{hi} + s \cdot W_{lo}$ where W_{hi} is FP8 and W_{lo} is 4-bit) has direction-dependent residual magnitude. Gradients in directions where $|W_{lo}|$ is small indicate the decomposition is "happy" there; gradients in directions where $|W_{lo}|$ is large indicate the decomposition is "unhappy." Sculpting gradients to attenuate the unhappy directions during the optimizer step keeps training stable even at very aggressive precision.

Port to NSL:

- CTFP's W_{hi}/W_{lo} decomposition is the direct analog to TCFP's.
- The output head's gradient is the primary target (highest sensitivity to quantization error).
- NSL-DIGS runs at the end of each backward pass: compute the confidence mask from W_{lo} magnitude, apply attenuation to output-head gradients before the optimizer step.

Pseudocode (simplified):

```
def nsl_digs_on_backward(grad, W_hi, W_lo,
    # Confidence mask: high where |W_lo| is small
    confidence = 1.0 - normalize(abs(W_lo))

    # Attenuate gradient in decomposition-unhappy directions
    sculpted_grad = grad * confidence_broadcast(confidence)

    return sculpted_grad
```

The implementation is roughly 200-300 LOC in NSL's backward pass for layers with CESH LM head role. The compute overhead is 8 element-wise operations per weight parameter per step — negligible.

DIGS is particularly effective at the output head because:

1. The output head's weight matrix has direct-to-vocabulary structure that exposes tokenizer biases in the gradient signal.
2. Output-head gradients are the largest in absolute magnitude (the loss signal concentrates here).
3. The cost of occasional wasted gradient direction is highest at the output head (a mislearned output distribution affects every downstream generation).

2.5 Innovation 5: Low-rank embedding decomposition

For models where the embedding matrix dominates memory (Gemma-7B's 20% case), CESH offers an optional low-rank factorization:

$$\text{Embedding}[V, d] \approx A[V, r] \cdot B[r, d], \quad r \ll d$$

With rank $r = 256$ and $d = 3072$ (Gemma-7B sizes), the compression ratio is $(V \cdot d) / (V \cdot r + r \cdot d) \approx 11\times$ for $V = 256000$.

The factorization is produced at training time (best accuracy) or at inference-load time (weaker accuracy but no training change). The forward pass becomes:

```
# For token_id t:
embed = A[t, :] @ B      # [1, r] @ [r, d] = [1, d]
```

This is equivalent to the original lookup at the cost of an extra small matmul per token. Since the extra matmul is small ($r \times d$ at typical sizes), the latency overhead is negligible — and on GPU it is often amortized across the batch (the matmul becomes $[\text{batch}, r] \times [r, d]$).

Activation at the `@role("embedding")` annotation:

```
ns!
```

```
@role("embedding", decomposition=LowRank(rank=256))  
embedding_layer: Embedding<[vocab_size, d_model]>
```

The decomposition is trainable: A and B are optimized during fine-tuning or continued pre-training. For inference-only low-rank (no retraining), the decomposition comes from SVD of the original matrix with a reconstruction loss bound.

2.6 Innovation 6: Outlier-safe training hook (OSP-compatible)

Recent research (Outlier-Safe Pre-Training, arXiv:2506.19697) established that outliers are not inevitable — they arise from specific choices (Adam's diagonal bias, layer-norm scaling, embedding initialization). OSP demonstrates that:

- Replacing Adam with Muon eliminates Adam-induced outliers.
- Single-Scale RMSNorm prevents channel-wise amplification.
- Learnable embedding projection redistributes magnitude.

OSP trained a 1.4B-parameter model with near-zero excess kurtosis — effectively outlier-free — at only 2% training overhead vs Adam.

CESH provides compile-time hooks for OSP-style training:

```
ns!
```

```
@role("embedding", outlier_safe=true)  
embedding_layer: Embedding<[vocab_size, d_model]>  
    // Automatically inserts learnable projection layer after embedding lookup  
  
@role("body", optimizer="muon", norm="single_scale_rms")  
transformer_blocks: ...
```

With these hooks active, CESH's quantization defaults become more aggressive: an outlier-safe model admits 4-bit weights everywhere including embedding and head, because the outlier sensitivity that normally protects these layers is eliminated.

This creates a positive feedback loop: OSP training → no outliers → more aggressive quantization allowed → smaller deployed model → same accuracy.

3. Implementation Architecture

3.1 Pipeline position

CESH operates primarily in Stage 3 (annotation resolution) and Stage 4 (WGGO).

Stage 3 (annotation resolution):

1. Read `@role` annotations from AST.
2. Infer missing roles from model graph structure.
3. Attach role property to every layer node.

Stage 4 (WGGO):

1. Read role property per layer.
2. Apply role-specific default policy (embedding/lm_head/body).
3. Record in WGGO decision table.
4. If DIGS is active, insert DIGS gradient-sculpting op after the backward of `@role("lm_head")` layers.

Stage 5 (MIR lowering):

1. If LM head rotation is active, apply Hadamard transformation to weights (offline, compile-time).
2. Emit fused Hadamard-apply at the input to the LM head (fused into preceding RMSNorm where possible).
3. If low-rank embedding is active, emit the $A @ B$ factored computation.

Stage 7 (runtime):

1. DIGS gradient sculpting runs during backward.
2. OSP-style Muon optimizer update if `optimizer="muon"` is set.

3.2 Role inference heuristics

The role-inference pass analyzes the model graph:

1. Walk the graph from input to output.
2. First `Embedding<...>` operation $\rightarrow$ role "embedding".
3. Final `Linear<[*, vocab_size]>` operation $\rightarrow$ role "lm_head".
4. If embedding and lm_head share weights (tied embedding) $\rightarrow$ mark both with shared role.
5. Everything between $\rightarrow$ role "body".

Edge cases:

- Multiple embedding-like layers (e.g., position embeddings) → only token embedding gets "embedding"; position embedding gets "auxiliary".
- Models without an explicit LM head (e.g., encoder-only) → no "lm_head" role; top layer gets "body".
- Fine-tuned models with added heads (classification, regression) → default to "body"; user can override.

3.3 DIGS integration with CFTP backward

DIGS requires access to the W_{lo} component of the CFTP decomposition during backward. CFTP stores W_{lo} as a persistent parameter in the optimizer state; the backward pass reads it as part of the standard gradient computation.

CESH adds a DIGS hook in the backward graph for `@role("lm_head")` layers:

```
# Standard CFTP backward
W_hi_grad, W_lo_grad = cftp_backward(output_grad, ...)

# DIGS sculpting (CESH insert)
if is_lm_head:
    confidence = 1.0 - normalize(abs(W_lo))
    W_hi_grad = W_hi_grad * confidence
    W_lo_grad = W_lo_grad * confidence

# Standard optimizer step
optimizer.step(W_hi_grad, W_lo_grad)
```

The hook fuses into the backward kernel emitter; no separate kernel launch.

3.4 Low-rank embedding runtime

When `decomposition=LowRank(rank=r)` is active, the embedding lookup path becomes:

```
# Original: embed = E[token_ids] (shape [batch, seq, d_model])
# CESH: embed = A[token_ids] @ B (shape [batch, seq, d_model])

# Phase 1: row lookup from A ( $V \times r$ )
embed_factor = A[token_ids] # [batch, seq, r]

# Phase 2: small matmul against B ( $r \times d_{model}$ )
embed = embed_factor @ B # [batch, seq, d_model]
```

The factored forward is batched efficiently on GPU. For typical $\text{batch} \times \text{seq} = 4 \times 2048 = 8192$ tokens and $(r, d) = (256, 4096)$, the factored matmul is $8192 \times 256 \times 4096 = 8.6$ GFLOPs — a few hundred microseconds on H100. Standard embedding lookup is $O(\text{batch} \times \text{seq} \times d_{\text{model}})$ memory accesses with no compute; the factored version adds compute but maintains similar memory bandwidth. Net impact: usually within 2% latency of the standard path.

4. Integration with NSL Pipeline

4.1 New invariants

I-32 — Layer roles are resolved at Stage 3 before WGGO. Role inference runs after annotation resolution; WGGO reads roles as inputs.

I-33 — Role-specific defaults can be overridden by explicit annotations. `@quant_override` or explicit `@cfip_format` on a layer takes precedence over the role's default policy.

I-34 — DIGS gradient sculpting requires CFTP decomposition present. DIGS cannot operate on layers quantized in a non-CFTP scheme. If LM head is CFIP-only (no CFTP), DIGS is inactive.

I-35 — Low-rank embedding factorization is opt-in. The default embedding representation is the standard lookup table. Low-rank is used only when explicitly annotated.

I-36 — Role-specific policies compose with CTQS phases. A CTQS phase transition can tighten body-layer bit budgets (e.g., from FP8 to FP4) without affecting embedding or head layer budgets.

4.2 Failure modes

F-21 — Role misinference. Symptom: the embedding layer gets body policy and quantizes aggressively; accuracy collapses. Root cause: role inference failed (ambiguous model structure). Mitigation: inference pass emits a warning when role is ambiguous; user can add explicit `@role("...")` to disambiguate.

F-22 — DIGS interacts with non-CFTP LM head. Symptom: compile-time error ("DIGS requires CFTP decomposition"). Root cause: user annotated `@digs` on an LM head that CFTP doesn't target. Mitigation: clear error message.

F-23 — Low-rank embedding accuracy collapse. Symptom: perplexity increase after switching to low-rank. Root cause: rank too low for the vocabulary's effective rank. Mitigation: CESH provides a rank-selection pass that SVD-decomposes the embedding and suggests a rank that captures 99%+ of the spectral energy.

5. Evaluation Plan

5.1 Baselines

1. **Uniform W4A4 (naive):** apply 4-bit everywhere; the catastrophic-failure baseline.
2. **AWQ:** industry baseline with first/last layer exemption.
3. **GPTQ with per-layer calibration.**
4. **QuaRot / SpinQuant with rotation.**
5. **CESH (this work):** with role-specific defaults.
6. **CESH + DIGS (this work):** with DIGS sculpting at LM head.
7. **CESH + OSP training:** with outlier-safe pre-training.

5.2 Benchmarks

- **Perplexity on wikitext-2** at W4A4 across baselines.
- **MMLU** for downstream accuracy at W4A4.
- **HumanEval** for code generation (relevant for NSLCoder).
- **Embedding-intensive: Gemma-7B at W4A4** — test where embedding+head is 20% of weights.

5.3 Specific accuracy targets

- Naive W4A4 baseline: typically 30-60% perplexity increase on wikitext-2.
- CESH role-specific: <5% perplexity increase.
- CESH + DIGS: <2% perplexity increase.
- CESH + DIGS + OSP: within 1% of FP16 baseline.

5.4 Correctness tests

1. **Role inference correctness** — verified against hand-labeled architectures.
2. **DIGS numerical equivalence vs no-DIGS** at FP16 (DIGS should be a no-op when no decomposition error exists).
3. **Low-rank reconstruction error** — SVD reconstruction error vs declared rank bound.
4. **Hadamard rotation invariance** — pre- and post-rotation LM head produce identical FP32 logits.

5.5 Implementation phases

Phase	Scope	Sessions
1	<code>@role</code> annotation + inference pass	1
2	Role-specific WGGO defaults	1
3	LM head Hadamard rotation	1
4	DIGS port from TCFP	2
5	Low-rank embedding forward path	1
6	Low-rank embedding training path	1
7	OSP hooks (Muon optimizer binding, SSNorm)	1
8	Correctness and benchmark tests	1
Total		9

Estimated scope: ~1550 LOC, 9 implementation sessions.

6. Discussion

6.1 Why this is the smallest paper in scope

CESH does less novel work than CCR, CKVQ, CGAC, or CSS because the core observations are well-established industry practice. CESH's contribution is making that practice a first-class NSL feature rather than user-applied convention.

The most novel component is the DIGS port — but DIGS itself is already a researched technique from CodeForge/TCFP. Porting to NSL is translation rather than invention.

6.2 Why it is still important

Naive quantization pipelines that ignore role produce high-profile accuracy failures (the "W4A4 just killed my model" class). CESH eliminates that failure mode by making it impossible to accidentally apply body policy to the LM head. Eliminating entire classes of user-error is the kind of work compilers should do.

6.3 Dependency on OSP

The most aggressive CESH configuration (4-bit everywhere, no role exemption) depends on outlier-safe pre-training. OSP itself is an active research area (2025 work); CESH's hooks assume OSP-compatible training hyperparameters are available. For models trained with standard Adam and no OSP hooks, CESH's conservative role-specific policy applies.

7. Conclusion

CESH formalizes the well-established industry practice of treating embedding and output-head layers differently from transformer body layers. It elevates layer role to a first-class compiler property, provides role-specific quantization defaults, ports the DIGS gradient-sculpting technique from TCFP, offers optional low-rank embedding decomposition, and integrates with outlier-safe pre-training hooks. The paper is intentionally smaller in novel technical content than the other four gap papers — CESH's value is eliminating an entire class of quantization accuracy failures by making the correct practice the default.

References

1. Dettmers, T. et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. NeurIPS 2022.
2. Lin, J. et al. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. arXiv:2306.00978.
3. Frantar, E. et al. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR 2023.
4. Ashkboos, S. et al. (2024). *QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs*. NeurIPS 2024.
5. Liu, Z. et al. (2024). *SpinQuant: LLM Quantization with Learned Rotations*. arXiv:2405.16406.
6. Xiao, G. et al. (2023). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. ICML 2023.
7. Park, J. et al. (2025). *Outlier-Safe Pre-Training for Robust 4-Bit Quantization of Large Language Models*. arXiv:2506.19697.
8. Wiemer, B. (2025). *DIGS: Decomposition-Informed Gradient Sculpting for TCFP-12*. CodeForge technical report.
9. Sun, M. et al. (2024). *Massive Activations in Large Language Models*. arXiv:2402.17762.
10. Bondarenko, Y. et al. (2023). *Quantizable Transformers: Removing Outliers by Helping Attention Heads Do Nothing*. NeurIPS 2023.